Introduction

{badge-virtual-threads} {badge-http2} {badge-groovy}

Overview

The **{project-name}** is a modern, production-grade HTTP client library for Groovy and Java applications. Built on Java's `HttpClient` API and leveraging virtual threads from Project Loom, it provides a powerful, fluent DSL for making HTTP requests with enterprise-grade features including circuit breakers, retry policies, security controls, and comprehensive testing support.

> A Groovy-idiomatic HTTP client that doesn't compromise on performance, security, or testability.

What Makes This Client Different?

Traditional HTTP clients often require choosing between ease of use and performance, or between rich features and simplicity. The Groovy HTTP Client eliminates these trade-offs:

Groovy DSL	Natural, expressive syntax that feels like writing configuration, not code
Virtual Thread Architecture	Millions of concurrent requests with minimal resource overhead
Security by Default	Comprehensive security controls with environment-aware profiles
Testing First	Purpose-built testing framework with mock server and fluent assertions
Production Ready	Circuit breakers, retry policies, metrics, and structured logging
Zero Magic	Wraps Java's standard `HttpClient` — no reflection, no bytecode manipulation

Key Features

Client Features

- **Fluent Groovy DSL** for request configuration, headers, and parameters
- **Automatic cookie management** with configurable policies
- **Multipart form uploads** via DSL or programmatic API
- **Circuit breaker** for resilience (CLOSED → OPEN → HALF_OPEN states)

- **Retry policies** with exponential backoff and jitter
- **Virtual thread executor** for ultra-lightweight concurrency
- **Request/response interceptors** for logging, metrics, and custom processing
- **Comprehensive security model** via `SecurityConfig` profiles
- **SSRF protection** with host allow-listing
- **HTTP/2 support** with automatic protocol negotiation

Testing Features

- **MockHttpServer** — Lightweight, dependency-free test server
- **HttpClientTestDsl** — Fluent DSL for endpoint mocking and verification
- **HttpClientTestTrait** — Spock trait for seamless test integration
- **Request recording** with rich assertion capabilities
- **Multipart validation** for file upload testing
- **Header and cookie verification** built-in

Requirements

Requirement	Version
Java	21 or higher (for virtual threads)
Groovy	5.0 or higher (tested with 5.0.4)
Build Tool	Gradle 9+ or Maven 3.8+

NOTE	Virtual threads are a preview feature in Java 19-20 and production-ready in Java 21+.

Quick Start

Add the dependency to your project:

build.gradle

```
dependencies {
    implementation '{project-group}:{project-artifact}:{project-version}'
}
```

Create a client and make your first request:

if you want to just try out with a public REST endpoint you can try https://restful-api.dev/ which has a list of resources and expected responses if you open that site with a browser.

When you create a 'new GroovyHttpClient ("https://<baseUrl>")', this internally creates a

SecurityConfig for you using SecurityConfig.testing(baseUrl). This is usually fine to get started, but you'll normally want to select from the SecurityConfig static methods or use the SecurityConfig builder for full control of the security permissions for your endpoint

Both synchronous and asynchronous http method calls are supported, and return an HttpClientResponse or CompletableFuture<HttpClientResponse> as return value

```
import org.softwood.http.GroovyHttpClient

// Simple client with testing defaults
// dummy endpoint - replace with your chosen base URL
def client = new GroovyHttpClient("https://api.example.com")

/** this is equivalent to
 * SecurityConfig conf = SecurityConfig.testing("https://api.example.com")
 * GroovyHttpClient gc = new GroovyHttpClient(conf)
 *
 * For production its best to build the full SecurityConfig instance for your needs
 * and pass that for the constructor
 **/

// Make a GET request
def response = client.getSync("/users/123")

println "Status: ${response.statusCode}"
println "Body: ${response.body}"

// Access parsed JSON
def user = response.json
println "User name: ${user.name}"
```

Who Should Use This Library?

This library is designed for:

Groovy Developers	Who want idiomatic, expressive HTTP client code
API Integration Projects	Needing robust retry logic and circuit breakers
Microservices	Requiring high-concurrency HTTP communication
Testing Scenarios	Where mock servers and request verification are critical
Security-Conscious Teams	Needing fine-grained control over TLS, timeouts, and SSRF protection
Java Developers	Looking for a modern alternative to Apache HttpClient or OkHttp

Design Philosophy

The library follows these core principles:

1. **Groovy-First, Java-Compatible**: Leverage Groovy's strengths while remaining usable from Java
2. **Wrap, Don't Replace**: Build on Java's standard `HttpClient` rather than reinventing HTTP
3. **Security by Configuration**: Make secure defaults easy, insecure options explicit
4. **Test-Driven Design**: Testing capabilities are first-class, not afterthoughts
5. **Performance Through Simplicity**: Virtual threads eliminate need for complex async patterns
6. **Fail Fast**: Validation happens early, errors are clear and actionable

What's in This Guide

This guide is organized into three main sections:

Core Concepts (Chapters 2-7)

Covers the architecture, security model, core client features, and multipart uploads. Read these chapters to understand how to configure and use the client effectively.

Testing Framework (Chapters 8-10)

Describes the `MockHttpServer`, `HttpClientTestDsl`, and `HttpClientTestTrait` for comprehensive HTTP testing. Essential reading for writing robust tests.

Examples & Reference (Chapters 11-12)

Provides real-world examples, troubleshooting guidance, and API reference material.

Getting Help

Documentation	This guide and inline Javadoc/GroovyDoc
Source Code	github.com/woodmawa/groovy-HttpClient
Issue Tracking	GitHub Issues
Email Support	will.woodman.softwood@outlook.com

> **TIP** The source code is extensively commented and can serve as additional documentation.

License

This library is licensed under the Apache License 2.0. See the `LICENSE` file for details.

```
Copyright 2025 Softwood Consulting Ltd
```

```
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
```

Next Steps

Continue reading to learn about:

- Architecture & Design Patterns — Understanding the virtual thread foundation
- Security Configuration — Environment profiles and security controls
- URL Builder — Constructing complex URLs with query parameters
- GroovyHttpClient Deep Dive — Comprehensive client usage guide

Let's dive in!